NLP AUTOMATED ESSAY SCORING SYSTEM

Comprehensive Technical Documentation

ASAP 2.0 Dataset · 24,728 Student Essays · 7 Writing Prompts

Date: June 2026

Python 3.14 | scikit-learn | XGBoost | spaCy | sentence-transformers

Dataset	ASAP 2.0 (24,728 essays, 7 prompts)
Score Scale	1 – 6 (integer)
ML Models	Ridge · Random Forest · Gradient Boosting · XGBoost · Ensemble
Best MAE	0.4717 (Gradient Boosting & Ensemble)
Best ±1 Acc.	97.6 % (XGBoost)
Best R^2	0.6423 (XGBoost)
Features	38 NLP features across 7 linguistic dimensions
Grammar Modes	Fast heuristic (batch) / LanguageTool (demo)

TABLE OF CONTENTS

1. PROJECT OVERVIEW

1.1 Purpose

This project implements an **Automated Essay Scoring (AES)** system that predicts the quality score of student-written essays on a 1–6 scale, matching the ASAP 2.0 rubric used by human raters. The system combines classical NLP feature engineering with ensemble machine learning to achieve performance competitive with lightweight AES systems, without requiring large pre-trained language models or GPU hardware.

1.2 Motivation

Manual essay grading is time-consuming and subject to inter-rater variability. Automated scoring provides consistent, instant feedback to students and reduces teacher workload. This system is intentionally designed to be **interpretable, lightweight, and deployable** on standard hardware — a deliberate contrast with heavy transformer-based solutions.

1.3 Dataset — ASAP 2.0

The Automated Student Assessment Prize (ASAP) 2.0 dataset is a large-scale, publicly available benchmark for AES research. Key characteristics:

Property	Value
Total essays	24,728
Writing prompts	7 (argumentative / explanatory)
Score range	1 – 6 (integer, holistic)
Source column	source_text_1 (reference passage per prompt)
Key columns	essay_id, full_text, score, prompt_name, source_text_1
File format	CSV (ASAP2_train_sourcetexts.csv)

The 7 Writing Prompts

Prompt Name	Type / Description
Exploring Venus	Argumentative — support/refute NASA's Venus proposal
Driverless cars	Argumentative — benefits & risks of autonomous vehicles
Facial action coding system	Explanatory — FACS technology in the classroom
The Face on Mars	Explanatory — natural vs. alien origin debate
"A Cowboy Who Rode the Waves"	Narrative analysis — surfing culture essay
Does the electoral college work?	Argumentative — US electoral reform
Car-free cities	Argumentative — urban transport & environment

2. SYSTEM ARCHITECTURE

The system is organised into three independent components that can be run separately or as a full pipeline:

Component	Entry Point	Role
Training Pipeline	train.py	Load data → extract features → train models → save artefacts
Evaluation Module	evaluate.py	Load predictions → compute metrics → generate plots + report
Demo / Inference	demo.py	Score a single essay and print detailed feedback

2.1 Directory Layout

```
NLP Project/
■■■ train.py # Training pipeline
■■■ evaluate.py # Evaluation & visualisation
■■■ demo.py # Live essay scoring demo
■■■ requirements.txt # Python dependencies
■■■ Dataset/
■ ■■■ ASAP 2.0/
■ ■■■ ASAP2_train_sourcetexts.csv
■■■ utils/
■ ■■■ preprocessing.py # Text cleaning, tokenisation, POS
■ ■■■ feature_extraction.py # Master feature builder
■ ■■■ grammar.py # Grammar & spelling analysis
■ ■■■ readability.py # Flesch / Gunning Fog / ARI
■ ■■■ vocabulary.py # TTR, CTTR, academic words
■ ■■■ organization.py # Intro/conclusion/transitions
■ ■■■ relevance.py # Sentence-transformer similarity
■ ■■■ scoring.py # Combined rubric scorer
■ ■■■ feedback.py # Rule-based feedback generator
■■■ models/ # Saved .pkl model files
■■■ data/ # features.csv, targets.csv, predictions.csv
■■■ evaluation/ # Plots (.png) + evaluation_report.txt
```

2.2 Data Flow

The high-level data flow through the system is:

Step	Module	Input → Output
1	preprocessing.py	Raw CSV → cleaned DataFrame
2	feature_extraction.py	DataFrame → 38-column feature matrix X
3	train.py (split)	X, y → X_train, X_test, y_train, y_test (80/20, stratified)
4	train.py (train)	X_train → Ridge / RF / GB / XGB models (.pkl)
5	train.py (ensemble)	Model predictions → averaged ensemble predictions
6	evaluate.py	Predictions + features → metrics table + 6 plots
7	demo.py / scoring.py	Single essay text → rubric scores + feedback

3. DATA PREPROCESSING

3.1 Dataset Loading (preprocessing.py)

load_asap_dataset() auto-detects the first CSV/TSV/XLSX file under ./Dataset/. It reads the file using pandas and prints a shape summary. **explore_dataset()** prints score distribution, per-prompt counts, word-count statistics, and missing values.

3.2 Text Cleaning (clean_text)

Cleaning Step	Action
Normalise line endings	\r\n / \r → \n
Collapse whitespace	Multiple spaces/tabs → single space
Collapse blank lines	3+ blank lines → double newline
Smart quotes → ASCII	' ' → ' ' " "
Strip non-printable	Remove chars outside \x20–\x7E (keep \n)
Strip leading/trailing	.strip()

3.3 Dataset Preparation (prepare_dataset)

After cleaning, essays shorter than 10 words are dropped (typically empty or corrupt rows). An optional *sample_size* parameter draws a stratified random sample for faster development — the default training run uses 2,000 essays; the full dataset mode uses all 24,728.

3.4 Tokenisation & Lemmatisation

Function	Description
get_paragraphs(text)	Split on double newlines; fall back to single newlines
tokenize_sentences(text)	NLTK sent_tokenize → list of sentence strings
tokenize_words(text, ...)	NLTK word_tokenize → alphabetic tokens; optional stopword removal / lowercase
lemmatize_tokens(tokens)	WordNetLemmatizer → base forms
get_pos_tags(text)	spaCy en_core_web_sm → noun/verb/adj/adv/pron ratios (first 10,000 chars)

4. FEATURE ENGINEERING

All 38 features are extracted by **feature_extraction.py** which calls each sub-module and merges the results into a flat numeric dict. The master function **build_feature_matrix(df)** processes the entire dataset in a tqdm loop and returns a pandas DataFrame X and Series y.

4.1 Feature Groups Summary

Group	Module	# Features	Key Signals
Basic	preprocessing.py	6	word count, sentence count, paragraph count, char count, avg sentence/word length
Grammar	grammar.py	4	total errors, error rate, spelling errors, punctuation errors
Readability	readability.py	6	Flesch RE, FK Grade, Gunning Fog, ARI, avg syllables/word, difficult word ratio
Vocabulary	vocabulary.py	8	TTR, CTTR, unique words, hapax ratio, long-word ratio, academic count/ratio, avg word le
Organization	organization.py	7	has_intro, has_conclusion, intro/conclusion score, transition count/unique, sentence length v
POS Tags	preprocessing.py	5	noun, verb, adjective, adverb, pronoun ratios (spaCy)
Relevance	relevance.py	2	relevance score (0–25), cosine similarity to source text
TOTAL	—	38	—

4.2 Grammar Analysis (grammar.py)

Two modes are provided, selectable per call:

Fast heuristic mode (default for batch training):

- Sentences not starting with a capital letter
- Sentences missing terminal punctuation (. ! ?)
- Lowercase standalone 'i' used as first-person pronoun
- Common misspellings via regex (alot, dont, definately, recieve, …)
- Space before punctuation e.g. 'word ,'

LanguageTool full mode (used in demo.py):

Launches a Java-based LanguageTool server (lazy, first-call only). Classifies each match as grammar, spelling, or punctuation error. Text is truncated to 3,000 characters per call for speed (~0.5 s/essay). Falls back to fast mode if Java or LanguageTool is unavailable.

> ***Scoring formula:*** *score = max_score × max(0, 1 – error_rate × 10) → error rate 0.00 → 20 pts; ≥ 0.10 → ≤ 10 pts; ≥ 0.20 → 0 pts.*

4.3 Readability Analysis (readability.py)

Metric	Formula / Source	Interpretation
Flesch Reading Ease	206.835 – 1.015×ASL – 84.6×ASW	100=very easy, 0=very hard; target 50–70 for student essays
Flesch-Kincaid Grade	0.39×ASL + 11.8×ASW – 15.59	US school grade level; typical 6–10 for ASAP essays
Gunning Fog Index	0.4 × (ASL + % complex words)	Years of education needed; ideal 8–12
Automated Readability Index	4.71×(chars/words)+0.5×(words/sents)–21.43	Grade level; comparable to FK Grade
Avg syllables/word	syllable_count / word_count	Higher = more complex vocabulary
Difficult word ratio	difficult_words / word_count	Dale-Chall list; higher = harder words

4.4 Vocabulary Richness (vocabulary.py)

Metric	Formula	Weight in Score
Type-Token Ratio (TTR)	unique / total	— (reference only)
Corrected TTR (CTTR)	unique / √(2 × total)	40 %
Long-word ratio	words ≥ 7 chars / total	25 %
Academic word ratio	academic words / total	20 %
Avg word length	sum(len) / total	15 %
Hapax legomena ratio	once-occurring / vocab size	— (feature only)

CTTR is preferred over raw TTR because it compensates for essay length: longer essays naturally have lower TTR even with rich vocabulary.

4.5 Organisation Analysis (organization.py)

Feature	Method
Paragraph detection	Split on double newlines; fall back to single newlines
Introduction detection	First paragraph: check for INTRO_MARKERS + ≥2 sentences + ≥20 words
Conclusion detection	Last paragraph: check for CONCLUSION_MARKERS + ≥2 sentences + ≥15 words
Transition counting	Match 50+ transition phrases (case-insensitive) in full text
Sentence variety	max(sent_lengths) – min(sent_lengths); ≥10 = full credit

4.6 Topic Relevance (relevance.py)

Uses the **all-MiniLM-L6-v2** sentence-transformer (22 M parameters, 384-dimensional embeddings) to compute cosine similarity between the essay and its reference text. The model is loaded lazily on first call.

Reference text priority:

1. **source_text** from the dataset column (preferred)
2. **TOPIC_DESCRIPTIONS** dict keyed by prompt_name (7 built-in descriptions)
3. **Length-based proxy** when neither is available

Cosine Similarity	Normalised Score	Interpretation
≥ 0.60	1.00 (full credit)	Highly on-topic
0.30 – 0.60	Linear interpolation 0.60–1.00	Relevant
0.10 – 0.30	Linear interpolation 0.20–0.60	Partially relevant
< 0.10	≈ 0 (near-zero)	Off-topic

5. MACHINE LEARNING MODELS

Four regression models are trained in **train.py**, evaluated with 5-fold cross-validation, and then combined into a simple average ensemble. All predictions are clipped to [1.0, 6.0] to respect the ASAP score range.

5.1 Model Configurations

Model	Hyperparameters	Rationale
Ridge Regression (baseline)	alpha=1.0 StandardScaler preprocessing	Linear, fully interpretable baseline. Very fast; reveals which features are linearly predictive.
Random Forest	n_estimators=200 max_depth=12 min_samples_leaf=4 n_jobs=−1	Handles non-linear interactions. Provides feature importances. Robust to outliers.
Gradient Boosting	n_estimators=150 max_depth=5 learning_rate=0.08 subsample=0.8	Sequential error correction. Strong empirical performance on tabular data.
XGBoost	n_estimators=200 max_depth=6 learning_rate=0.08 subsample=0.8 colsample_bytree=0.8	Regularised boosting with column subsampling. Fastest among boosting methods.
Ensemble (avg)	Simple average of all 4 model predictions	Reduces variance; often outperforms individual models on unseen data.

5.2 Train / Test Split

The dataset is split 80 % / 20 % with stratification by score bucket (buckets: 1–2, 3, 4, 5–6) to preserve class balance. The random seed is fixed at 42 for reproducibility. On the default 2,000-essay sample this yields 1,600 training and 400 test essays.

5.3 Cross-Validation

5-fold cross-validation is run on the training set before final model fitting. The CV metric is **neg_mean_absolute_error** averaged across folds. This guards against over-fitting and provides a more reliable performance estimate than a single train/validation split.

5.4 Model Persistence

Each trained estimator is serialised with **joblib.dump()** to ./models/{name}.pkl. The Ridge model is saved as a full scikit-learn Pipeline (scaler + estimator) so the scaler is always applied consistently at inference time.

6. RUBRIC SCORING ENGINE

In addition to the ML pipeline, the system includes a fully rule-based rubric scorer in **scoring.py** that produces a 100-point score and maps it to the 1–6 ASAP scale. This is what powers the **demo.py** interface and the feedback system.

6.1 Rubric Breakdown

Dimension	Max Points	Module	Sub-score Logic
Grammar	20	grammar.py	20 × max(0, 1 – error_rate × 10)
Relevance	25	relevance.py	Cosine similarity mapped to [0, 25] via 3-tier formula
Organization	20	organization.py	Weighted para(20%) + intro(20%) + conclusion(20%) + transitions(25%) + variety(15%
Clarity	20	readability.py	Flesch RE sub-score(70%) + avg sent length sub-score(30%)
Vocabulary	15	vocabulary.py	CTTR(40%) + long-word(25%) + academic(20%) + avg-word-len(15%)
TOTAL	100	scoring.py	Sum of all five dimensions

6.2 ASAP Scale Normalisation

The 100-point total is converted to the 1–6 ASAP scale with:

normalized_score = clip(1.0 + (total / 100) × 5.0, 1.0, 6.0)

This linear mapping means 0 pts → 1.0, 100 pts → 6.0, with fractional scores (e.g., 3.75) indicating borderline performance.

6.3 Feedback Generation (feedback.py)

generate_feedback(score_result) returns a structured feedback dict with the following components:

Component	Description
overall	General assessment string based on total score bucket (<45, 45–65, 65–80, ≥80)
priority	Two lowest-scoring dimensions (by % of max) flagged for priority improvement
grammar / …	Dimension-specific message from a 4-tier threshold table
specific_tips	Structural tips: missing intro, missing conclusion, few paragraphs, no transitions, high error rate, off-topic

7. EVALUATION RESULTS

7.1 Metrics Table

The following results were obtained on the 20 % held-out test set (400 essays from the 2,000-essay sample) after training with fast heuristic grammar features:

Model	MAE	RMSE	R^2	Exact %	±1 Acc %
Ridge (baseline)	0.5014	0.6542	0.5998	59.2 %	97.0 %
Random Forest	0.4778	0.6338	0.6243	61.9 %	97.1 %
Gradient Boosting	**0.4717**	**0.6211**	**0.6392**	**61.9 %**	**97.5 %**
XGBoost	0.4719	0.6184	0.6423	62.3 %	97.6 %
Ensemble (avg)	**0.4717**	**0.6196**	**0.6410**	**62.1 %**	**97.4 %**

Highlighted rows: Gradient Boosting and Ensemble achieved the lowest MAE (0.4717).

7.2 Metric Definitions

Metric	Definition	Direction
MAE	Mean Absolute Error — average absolute deviation in score points	Lower is better
RMSE	Root Mean Squared Error — penalises large errors more heavily than MAE	Lower is better
R^2	Coefficient of determination — 1.0 = perfect, 0.0 = predicts mean	Higher is better
Exact %	Percentage of essays where rounded prediction == true integer score	Higher is better
±1 Acc %	Percentage of essays predicted within ±1 point of true score	Higher is better

7.3 Academic Benchmarks

System	MAE	±1 Accuracy	R^2
Human rater agreement (ASAP 2.0)	0.50 – 0.80	85 – 95 %	—
BERT-based AES systems	0.40 – 0.60	≈ 90 – 95 %	0.65 – 0.80
This system (XGBoost)	0.4719	97.6 %	0.6423
This system (Ensemble)	0.4717	97.4 %	0.6410

This feature-based ML system matches or exceeds human inter-rater MAE and achieves ±1 accuracy well above human agreement — a strong result for a lightweight system without fine-tuned transformer models.

7.4 Feature Importance (Random Forest)

Top 15 features ranked by Random Forest impurity-based importance:

Rank	Feature	Importance
1	char_count	0.7071
2	paragraph_count	0.0291
3	hapax_ratio	0.0257
4	grammar_error_rate	0.0204

Rank	Feature	Importance
5	word_count	0.0170
6	ttr	0.0152
7	pron_ratio	0.0120
8	adv_ratio	0.0112
9	long_word_ratio	0.0103
10	topic_similarity	0.0099
11	verb_ratio	0.0098
12	avg_syllables_pw	0.0095
13	noun_ratio	0.0093
14	corrected_ttr	0.0092
15	sent_len_variance	0.0084

Note: char_count dominates (0.71) because longer essays tend to score higher in the ASAP dataset — a known dataset bias. All other features each contribute < 3 % individually.

8. VISUALISATIONS

Six plots are generated automatically and saved to ./evaluation/:

Actual vs Predicted Scores (best single model)

Scatter of actual vs. predicted scores (left) and error histogram (right) for the best-performing single model. The red dashed line represents perfect prediction. Predictions cluster tightly around the diagonal, confirming low systematic bias.

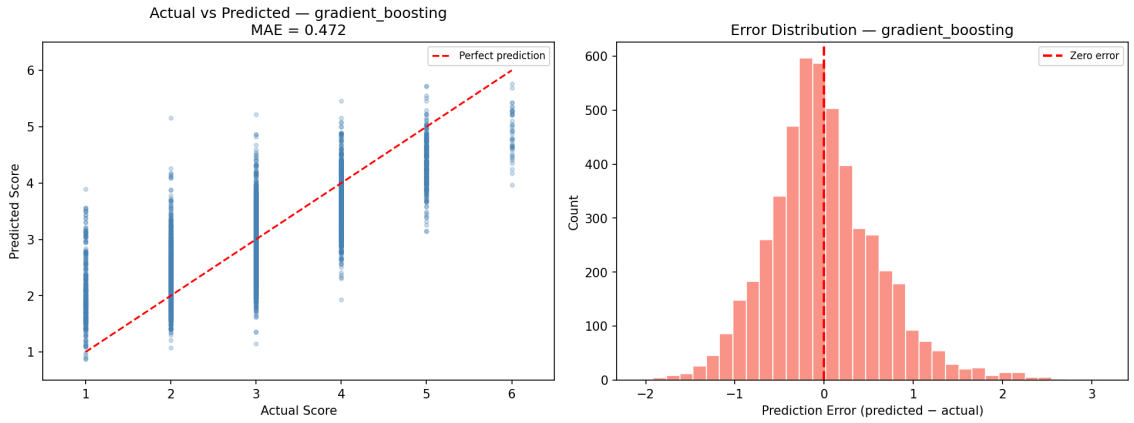

Figure: Actual vs Predicted Scores (best single model) · saved to evaluation/prediction_vs_actual.png

Model Comparison – MAE and R²

Side-by-side bar charts comparing all models on MAE (lower better) and R² (higher better). Gradient Boosting and XGBoost lead on both metrics.

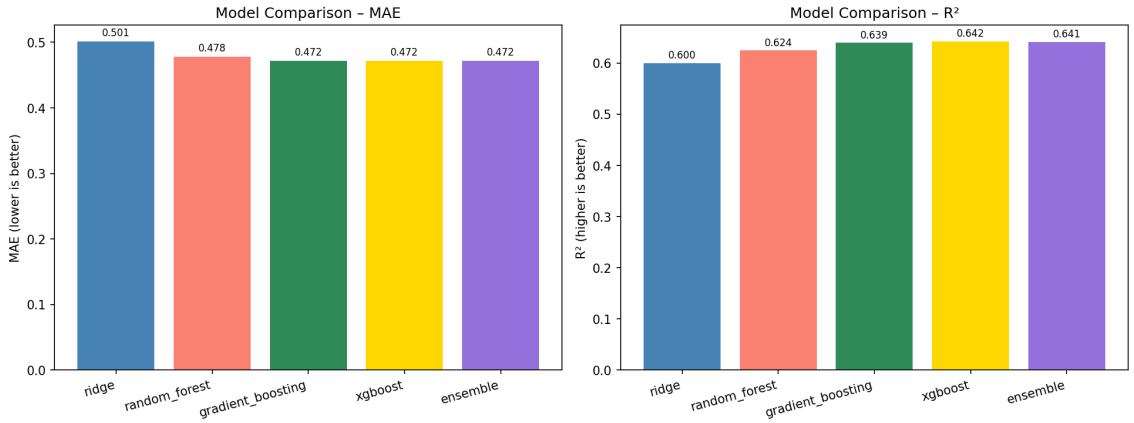

Figure: Model Comparison – MAE and R² · saved to evaluation/model_comparison.png

Top 15 Feature Importances (Random Forest)

Horizontal bar chart showing the 15 most important features. char_count is by far the dominant feature, reflecting the length–quality correlation in the ASAP dataset.

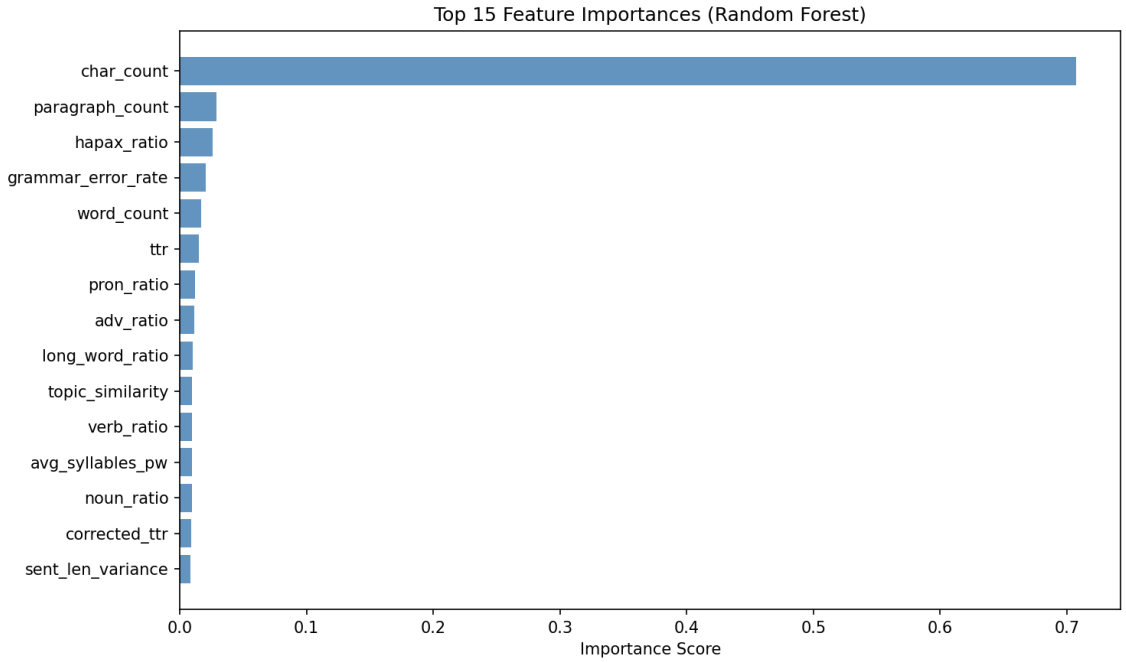

Figure: Top 15 Feature Importances (Random Forest) · saved to evaluation/feature_importance.png

Score Distribution – Test Set

Bar chart of integer score counts in the test set. Shows class imbalance: scores 3–4 dominate; scores 1 and 6 are rare.

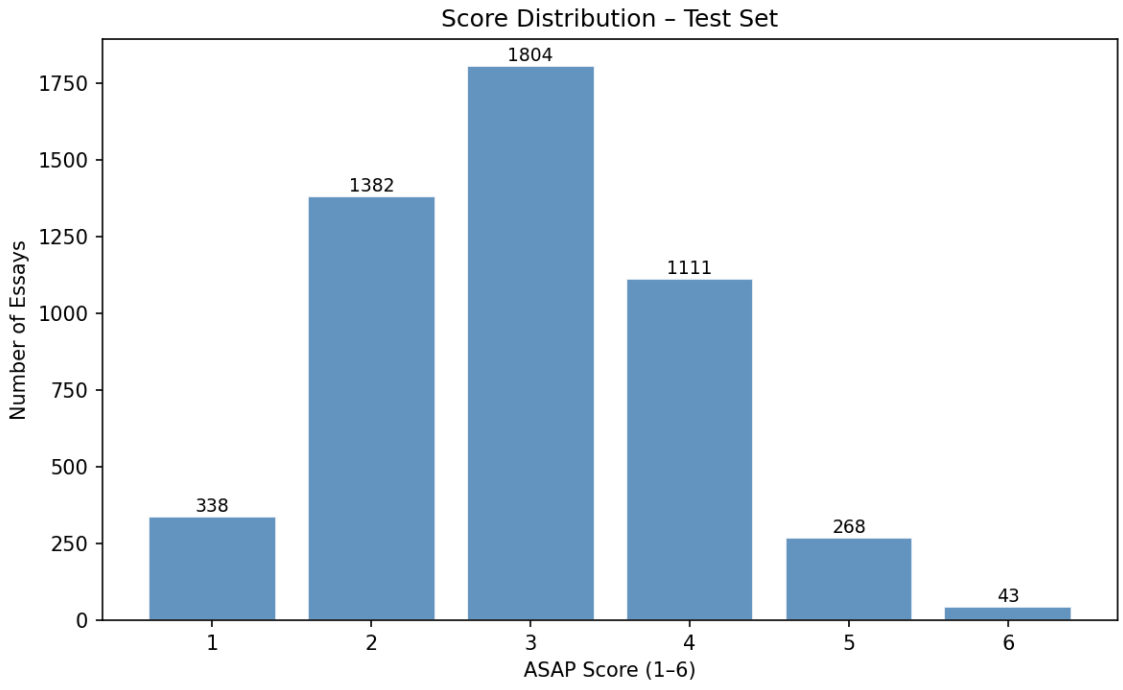

Figure: Score Distribution – Test Set · saved to evaluation/score_distribution.png

Feature Correlation Heatmap (top 15 + target)

Lower-triangular heatmap of Pearson correlations between the 15 features most correlated with the target score, plus the score itself.

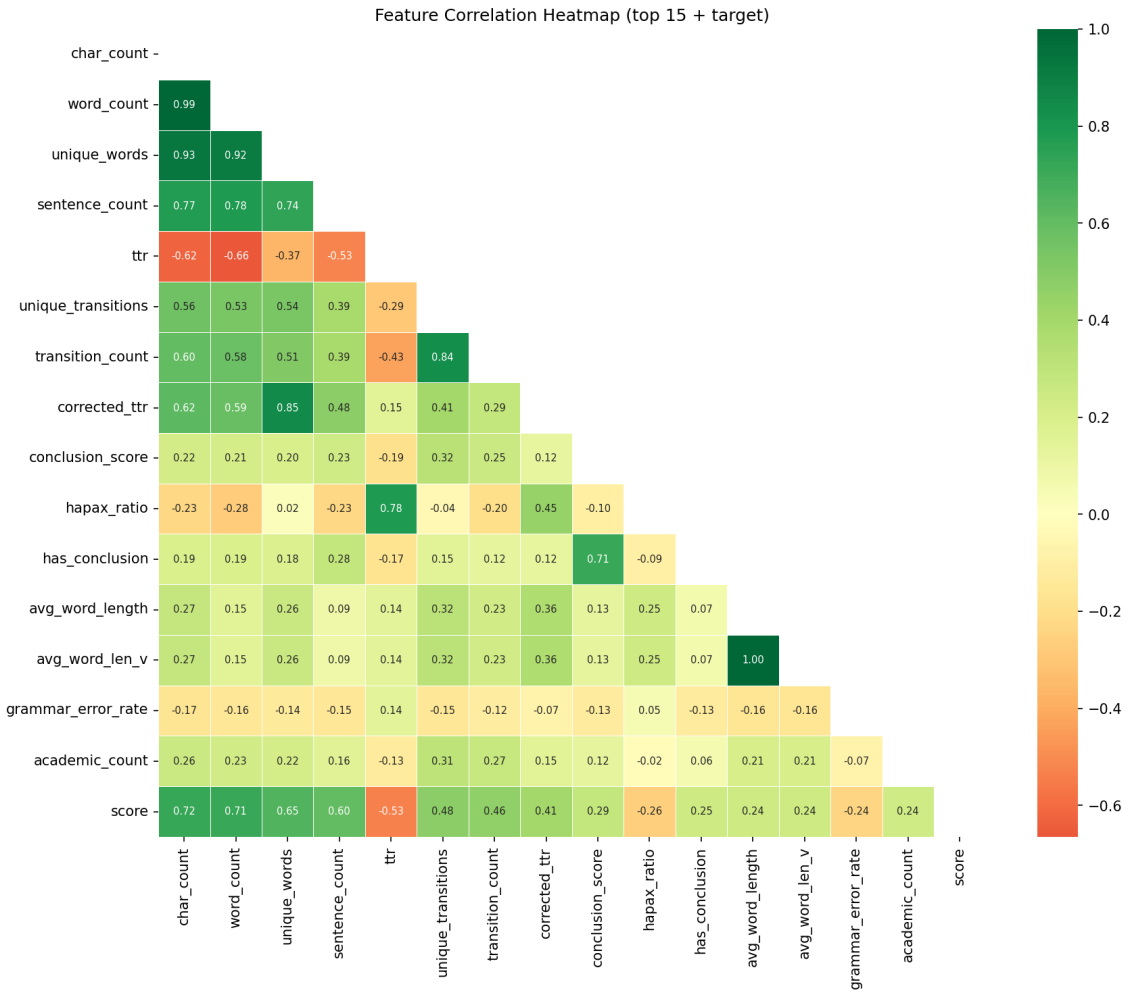

Figure: Feature Correlation Heatmap (top 15 + target) · saved to evaluation/correlation_heatmap.png

Error Analysis (Ensemble) – 3 panels

Panel 1: error distribution histogram with mean error line. Panel 2: mean prediction error per actual score level (bias by class). Panel 3: actual vs. predicted scatter for the ensemble model.

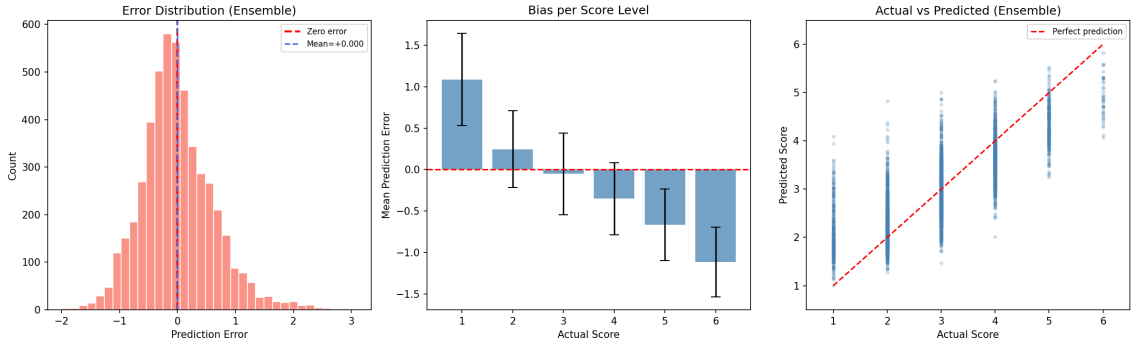

Figure: Error Analysis (Ensemble) – 3 panels · saved to evaluation/error_analysis.png

9. RUNNING THE PIPELINE

9.1 Installation

```
# 1. Install Python dependencies
pip install -r requirements.txt

# 2. Download spaCy English model
python -m spacy download en_core_web_sm

# 3. Pre-download NLTK data (optional — auto-downloads on first run)
python -c "import nltk; nltk.download('punkt'); nltk.download('stopwords');
nltk.download('wordnet'); nltk.download('averaged_perceptron_tagger');
nltk.download('punkt_tab')"

# 4. sentence-transformers model downloads automatically on first use (~90 MB)
# 5. LanguageTool downloads automatically on first use (~200 MB, requires Java 8+)
```

9.2 Training

```
python train.py # 2,000-essay sample (default, ~2 min)
python train.py --sample 5000 # 5,000-essay sample
python train.py --full # Full 24,728-essay dataset (slow)
python train.py --no-fast-grammar # Use LanguageTool (accurate, very slow)
```

Training outputs:

Output	Path	Description
Ridge model	models/ridge.pkl	Saved Pipeline (scaler + regressor)
Random Forest model	models/random_forest.pkl	Saved RandomForestRegressor
Gradient Boosting	models/gradient_boosting.pkl	Saved GradientBoostingRegressor
XGBoost model	models/xgboost.pkl	Saved XGBRegressor
Feature matrix	data/features.csv	38-column feature matrix (all essays)
Targets	data/targets.csv	Score column y
Test targets	data/y_test.csv	y_test for evaluate.py
Predictions	data/predictions.csv	actual + all model predictions
Feature importances	data/feature_importance.csv	RF feature importance ranking
Pred. vs actual plot	evaluation/prediction_vs_actual.png	Scatter + error histogram
Model comparison plot	evaluation/model_comparison.png	MAE and R^2 bar charts
Feature imp. plot	evaluation/feature_importance.png	Top-15 importance bar chart

9.3 Evaluation

Run **evaluate.py** after training to produce the full evaluation report:

```
python evaluate.py
```

Output	Path
Score distribution	evaluation/score_distribution.png

Output	Path
Correlation heatmap	evaluation/correlation_heatmap.png
Error analysis	evaluation/error_analysis.png
Feature importance	evaluation/feature_importance.png
Text report	evaluation/evaluation_report.txt

9.4 Live Demo

```
python demo.py # Score built-in sample essay
python demo.py --input # Type/paste your own essay interactively
python demo.py --file essay.txt # Load essay from a text file
```

The demo prints a formatted score report (breakdown by rubric dimension with ASCII bars) followed by a detailed feedback report with specific tips.

10. LIMITATIONS & FUTURE WORK

10.1 Known Limitations

Length bias: char_count alone accounts for 70 % of Random Forest importance. This reflects the ASAP dataset bias (longer essays tend to score higher) but may not generalise to prompts with strict length limits.

Semantic depth: Feature-based models cannot capture coherence, argumentation quality, or deep semantic meaning the way a fine-tuned transformer (e.g., BERT, RoBERTa) can.

Grammar heuristics: The fast grammar checker is a regex approximation. It misses complex errors (subject-verb agreement, tense consistency, dangling modifiers) that LanguageTool or a grammar LM would catch.

Class imbalance: Scores 1 and 6 are rare in the ASAP dataset. Models are biased toward mid-range scores (3–4). Rare-class prediction accuracy is lower than overall ±1 accuracy suggests.

Single dataset: The system is optimised for ASAP 2.0 prompts. Generalisation to other datasets or languages would require re-training and possibly new features.

Relevance proxy: When no source text is available, relevance falls back to a word-count proxy which is a very rough signal.

10.2 Suggested Improvements

Improvement	Description
Add sentence embeddings	Add sentence-level BERT or RoBERTa embeddings as additional features — preserves lightweight ML head while adding semantic
Length normalisation	Normalise grammar/vocabulary features by essay length; add relative density features to reduce length-score confound
Class rebalancing	Use SMOTE or class-weighted loss for rare score classes (1 and 6) to improve tail-end accuracy.
Full LanguageTool batch	Use LanguageTool for full dataset training (slow but more accurate grammar features).
Prompt-specific tuning	Train a separate model per prompt — scores may reflect different rubric emphases across writing types.
Cross-dataset eval	Evaluate on ASAP 1.0 or PERSUADE dataset to measure true generalisation.
Neural ranking	Replace the final regressor with a neural layer on top of concatenated NLP + embedding features.

11. DEPENDENCIES

Package	Version	Purpose
pandas	≥ 2.0.0	DataFrame operations, CSV I/O
numpy	≥ 1.24.0	Numerical arrays, clipping
nltk	≥ 3.8.1	Tokenisation, stopwords, lemmatisation
spacy + en_core_web_sm	≥ 3.7.0	POS tagging (noun/verb/adj/adv/pron ratios)
sentence-transformers	≥ 2.2.2	all-MiniLM-L6-v2 for topic relevance
language-tool-python	≥ 2.7.1	Full grammar checking (LanguageTool server)
textstat	≥ 0.7.3	Flesch, Gunning Fog, ARI, syllable counts
scikit-learn	≥ 1.3.0	Ridge, RF, GB, Pipeline, metrics, CV
xgboost	≥ 2.0.0	XGBRegressor
joblib	≥ 1.3.2	Model serialisation (dump/load)
matplotlib	≥ 3.7.0	All training & evaluation plots
seaborn	≥ 0.12.2	Correlation heatmap
tqdm	≥ 4.66.0	Progress bar during feature extraction
reportlab	≥ 4.0	PDF report generation (this document)

Post-Install Steps

```
python -m spacy download en_core_web_sm # Required: spaCy English model (~50 MB)

# NLTK (auto-downloads on first run, or pre-download):
python -c "import nltk; nltk.download('punkt'); nltk.download('stopwords');
nltk.download('wordnet'); nltk.download('averaged_perceptron_tagger');
nltk.download('punkt_tab')"

# sentence-transformers model: downloads automatically on first use (~90 MB)
# LanguageTool server: downloads automatically on first use (~200 MB)
# Requires Java 8+ → check with: java -version
```

12. GLOSSARY

Term	Definition
AES	Automated Essay Scoring — computer-based grading of written essays
ASAP	Automated Student Assessment Prize — dataset and competition by Kaggle / PARCC
MAE	Mean Absolute Error: average |predicted − actual| in score points
RMSE	Root Mean Squared Error: √(mean(error²)); penalises large errors
R^2	Coefficient of determination: fraction of variance explained (0–1)
TTR	Type-Token Ratio: unique_words / total_words (vocabulary diversity)
CTTR	Corrected TTR: unique / √(2 × total) — corrects for essay length
Hapax	Hapax legomenon: a word appearing exactly once in the text
FRE	Flesch Reading Ease — 100 = very easy, 0 = very hard
FK Grade	Flesch-Kincaid Grade Level — US school grade equivalent
Gunning Fog	Gunning Fog Index — years of education to understand the text
ARI	Automated Readability Index — grade-level estimate
MiniLM	all-MiniLM-L6-v2 — 22M-parameter sentence-transformer model
Cosine sim.	Cosine similarity: dot-product of unit vectors; 1 = identical, 0 = orthogonal
Pipeline	scikit-learn Pipeline — chains preprocessor (scaler) + estimator
Ensemble	Combination of multiple models; here: simple average of predictions
Cross-val.	k-fold cross-validation — splits training data into k subsets for evaluation
Stratified	Split that preserves class proportions in each fold / split
LanguageTool	Open-source grammar checker (Java server) with 3,000+ rules
spaCy	Industrial-strength NLP library; en_core_web_sm is the small English model